10/26

Exp: B Implementation of Ping Program.

Aim:

To implement a simple ping program to check if a destination host is reachable and to measure the round trip of packet.

Algorithm:

- Start the program, and read the destination host name or IP address from the user.
- Create a raw ICMP socket and echo request containing ICMP type, code, checksum, Identifier, sequence number, data.
- Calculate the checksum of the packet and send the ICMP echo request to the destination
- Wait for a reply, then calculate the RTT.
- Display the destination, packet size, sequence numbers, and RTT. Repeat for a specified number of packets.

Code :

```
import socket
import struct
import time
import select
import os
import statistics
ICMP_ECHO_REQUEST = 8
ICMP_ECHO_REPLY = 0
PACKET_SIZE = 64
TIMEOUT = 2
PACKET_COUNT = 4
def checksum(data):
    total = 0
    length = len(data)
    i = 0
    while i < length - 1:
        total += (data[i] << 8) + data[i + 1]
        i += 2
    if length % 2:
        total += data[-1] << 8
    while total >> 16:
        total = (total & 0xFFFF) + (total >> 16)
    return (~total) & 0xFFFF
def create_packet(packet_id, sequence):
    header = struct.pack(
        "!BBHHH",
        ICMP_ECHO_REQUEST, 0, 0,
        packet_id, sequence
```

```
def ping(host, sequence):
    try:
        destination = socket.gethostbyname(host)
        sock = socket.socket(
            socket.AF_INET, socket.SOCK_RAW,
            socket.IPPROTO_ICMP
        )
        sock.settimeout(TIMEOUT)
        packet_id = os.getpid() & 0xFFFF
        packet = create_packet(packet_id, sequence)
        start_time = time.time()
        sock.sendto(packet, (destination, 1))
        while True:
            ready = select.select([sock], [], [], TIMEOUT)
            if not ready[0]:
                print("Request timed out.")
                sock.close()
                return None
            received_packet, address = sock.recvfrom(1024)
            end_time = time.time()
            ip_header_length = (received_packet[0] & 15) * 4
            icmp_header = received_packet[ip_header_length:ip_header_length + 8]
            if len(icmp_header) < 8:
                continue
```

```
def main():
    host = input("Enter hostname or IP address: ")
    try:
        destination = socket.gethostbyname(host)
    except socket.gaierror:
        print("Invalid hostname or IP address.")
        return
    print(f"\nPinging {host} [{destination}]")
    print(f"Sending {PACKET_COUNT} ICMP packets:\n")
    results = []
    for sequence in range(1, PACKET_COUNT + 1):
        result = ping(host, sequence)
        if result is not None:
            results.append(result)
        time.sleep(1)
    sent = PACKET_COUNT
    received = len(results)
    lost = sent - received
    print("\n----- Ping Statistics -----")
    print(f"Packets: Sent = {sent}, Received = {received}, "
          f"Lost = {lost} ({lost / sent * 100:.0f}% loss)")
    if results:
        print("\n----- Round Trip Time -----")
        print(f"Minimum = {min(results):.2f} ms")
        print(f"Maximum = {max(results):.2f} ms")
        print(f"Average = {statistics.mean(results):.2f} ms")
```

Output :

```
Enter hostname or IP address: 8.8.8.8

Pinging 8.8.8.8 [8.8.8.8]
Sending 4 ICMP packets:

Reply from 8.8.8.8: bytes=56 time=24.35 ms
Reply from 8.8.8.8: bytes=56 time=23.82 ms
Reply from 8.8.8.8: bytes=56 time=25.16 ms
Reply from 8.8.8.8: bytes=56 time=24.02 ms

----- Ping Statistics -----
Packets: Sent = 4, Received = 4, Lost = 0 (0% loss)

----- Round Trip Time -----
Minimum = 23.82 ms
Maximum = 25.16 ms
Average = 24.34 ms
```

Result :

The implementation of Ping Program has been completed successfully.